Running Secure Agents at Scale

Kubernetes sandboxes, with Playwright as the workload

Adobe

Carlos Sanchez / csanchez.org / @csanchez.org

Carlos Sanchez

Principal Scientist

Adobe Experience Manager

Long-time OSS contributor:
Kubernetes, Jenkins, Apache Maven,
...

Your agent needs a browser

What else did you just give it?

Network access. Filesystem access. Credentials.

A long-lived process tree.

> A tool call is also a resource and trust-boundary decision.

Why Playwright?

- Chromium is stateful and multiprocess
- Native WebSockets exercise the whole data plane
- Startup and reuse are measurable
- A real page fetch gives us a correctness check

> A green controller Deployment is not a working browser.

Three separate decisions

Lifecycle

Who creates, reuses, suspends, and deletes the environment?

Runtime

What separates the workload from the host kernel and other workloads?

Policy

Who may connect, and what may the browser reach?

> Do not compare a warm pool with a kernel boundary.

01 / THE OPTIONS

Same workload. Different contracts.

Four current integrations

Backend	Unit / provisioning	Runtime in this setup	State after lifecycle action
agent-sandbox	Claim a warm sandbox	Pod runtime; configurable	Recreate: fresh browser
isola	Sandbox CR; on demand	gVisor	Recreate: fresh browser
substrate	Actor on a worker	gVisor	Explicit memory suspend/resume
Azure KARS	CR; dedicated namespace	Pod + governed networking	Recreate: fresh browser

Scope: the October 16, 2026 integration run. Versions and caveats are in the appendix. Source

agent-sandbox

Keep the browser warm. Claim it when needed.

- Kubernetes-native templates, warm pools, and claims
- Choose a configured pod runtime through `RuntimeClass`
- Reuse the sandbox while its claim is alive
- Pay for warm capacity before the request arrives

kubernetes-sigs/agent-sandbox / tested integration: v1.0.7

isola

An on-demand gVisor path

- A `Sandbox` CR creates a gVisor-isolated pod
- Shared sandbox namespace in this integration
- No warm pool in this adapter
- Fixture access needs an explicit network allowance

isola-run/isola / tested integration: v0.7.0

substrate

An actor lifecycle with memory resume

- API-managed actors run on gVisor workers
- Explicit suspend creates a durable memory snapshot
- A later request can trigger resume
- More control-plane and integration machinery

> Snapshot support matters only if the application resumes correctly.

agent-substrate/substrate / tested integration: v0.5.0

Azure KARS

A namespace per sandbox, with enforced egress

- Dedicated namespace and pod per `KarsSandbox`
- Namespace-scoped policy and resource boundaries
- Loopback browser rendering passed
- The private ClusterIP fixture fetch **failed**

> A domain allowlist did not override the tested private-IP restriction.

Azure/kars / tested integration: v0.1.26

And OpenShell?

The repository has a fifth backend.

In this harness, it uses the **SandboxClaim adapter** with a different sandbox image.

It is in the original comparison, but **not in the October pinned four-backend rerun**.

nvidia/openshell / Original comparison

Choose by constraint

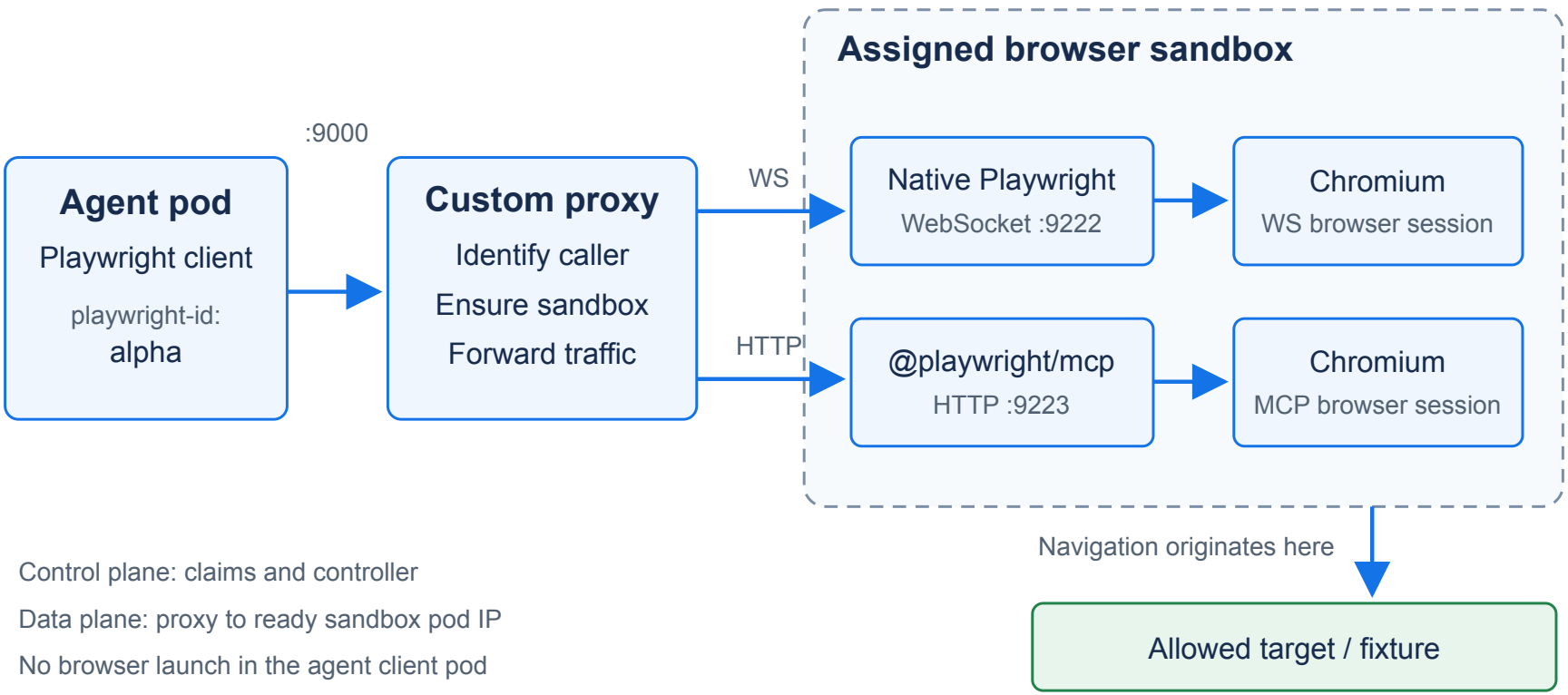
If you need...	Investigate...	Validate before committing
Fast assignment of prepared browsers	agent-sandbox	Warm capacity, runtime, and refill
On-demand pods with gVisor	isola	Runtime installation and network policy
Explicit memory suspend/resume	substrate	Application-state persistence oracle
Namespace-per-sandbox governance	KARS	Required targets under enforced egress

> Choose the lifecycle and boundary your workload actually needs.

02 / INSIDE AGENT-SANDBOX

Separate the agent from the browser

One endpoint, dedicated sandboxes



Application architecture: playwright-k8s-sandbox. This integration does not use upstream sandbox-router, sandboxd, or the SDKs.

Four objects to understand

Object	Responsibility
`SandboxTemplate`	Defines the workload pod
`SandboxWarmPool`	Keeps prepared sandboxes available
`SandboxClaim`	Requests a sandbox from the pool
`Sandbox`	Represents the managed workload

> The controller manages lifecycle. The proxy carries browser traffic.

Define the browser workload

```yaml
apiVersion: extensions.agents.x-k8s.io/v1beta1
kind: SandboxTemplate
metadata:
  name: playwright
  namespace: pw-test
spec:
  networkPolicyManagement: Unmanaged
  podTemplate:
    spec:
      automountServiceAccountToken: false
      containers:
        - name: playwright
          image: localhost:5001/playwright-substrate:slim-1.53.0
          command: ["node", "/server.js"]
```

Excerpt: complete harness template. Environment, probes, and limits omitted here. Image is preloaded into kind; replace it for a real registry.

Preboot a small pool

```yaml
apiVersion: extensions.agents.x-k8s.io/v1beta1
kind: SandboxWarmPool
metadata:
  name: playwright
  namespace: pw-test
spec:
  replicas: 3
  sandboxTemplateRef:
    name: playwright
  updateStrategy:
    type: OnReplenish
```

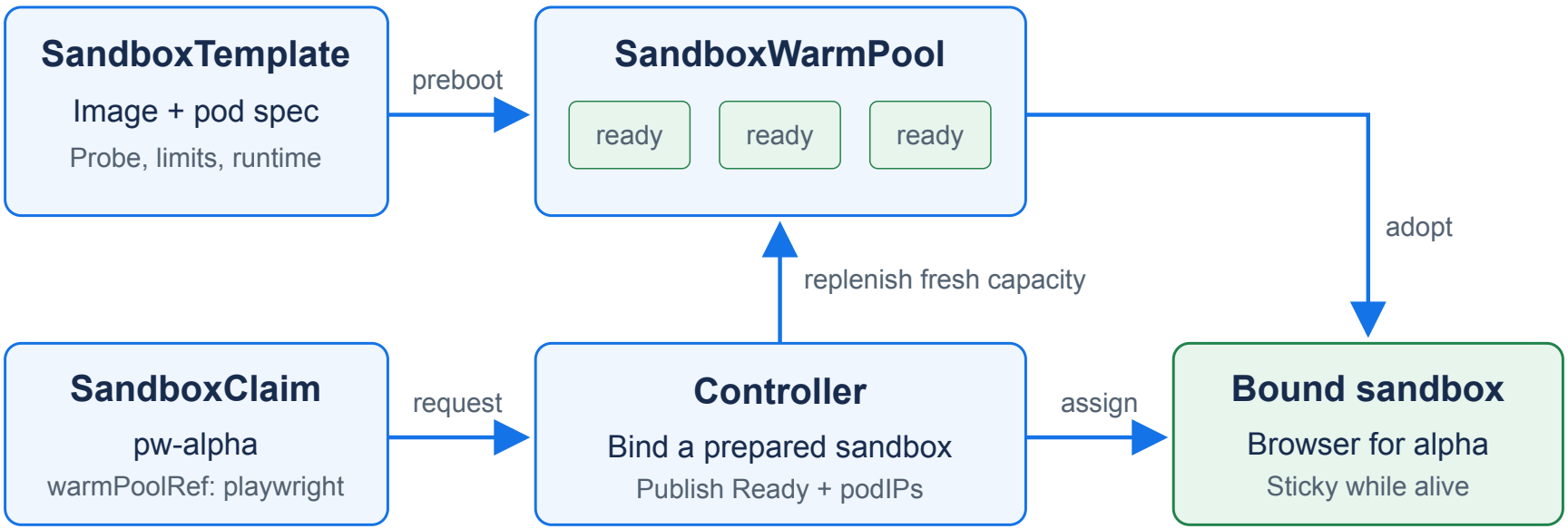
> Three prepared sandboxes, not a three-tenant concurrency limit.

Request one browser

```yaml
apiVersion: extensions.agents.x-k8s.io/v1beta1
kind: SandboxClaim
metadata:
  name: pw-alpha
  namespace: pw-test
  labels:
    playwright-id: alpha
    playwright-proxy/managed: "true"
spec:
  warmPoolRef:
    name: playwright
```

Claim shape from the adapter; additionalPodMetadata omitted. The proxy creates this resource.

Claim, bind, replenish



Delete claim -> destroy its workload. Next claim -> fresh browser, not restored memory.

> Prepared capacity is reusable infrastructure, not preserved tenant memory.

Ready means ready to use

The proxy waits for both:

```
status.conditions[type=Ready].status == "True"
status.sandbox.podIPs contains an address
```

The browser template probes **9222**, after Chromium starts.

The server waits for the MCP listener before starting WS.

> Pod phase Running is not application readiness.

How does the caller get its browser?

```yaml
# On the client Job's pod template:
spec:
  template:
    metadata:
      labels:
        playwright-id: alpha
```

Source pod IP -> pod label -> session ID -> `pw-alpha`

Same ID: same live sandbox. Different IDs: different sandboxes.

> A user-controlled label is not tenant authentication.

Two protocol paths

Client API	Client endpoint	Sandbox listener
`chromium.connect()`	`ws://playwright-proxy:9000/`	Native Playwright WS: **9222**
MCP streamable HTTP	`http://playwright-proxy:9000/mcp`	`@playwright/mcp`: **9223**

HTTP upgrade goes to WS.

Plain HTTP goes to MCP.

Native Playwright protocol, not CDP. WS and MCP use separate browser sessions inside the sandbox.

Playwright code stays familiar

```
import { chromium } from 'playwright';

const browser = await chromium.connect(
  'ws://playwright-proxy:9000/'
);
try {
  const context = await browser.newContext();
  const page = await context.newPage();
  const response = await page.goto(
    'http://test-target.pw-test.svc.cluster.local/'
  );
  if (!response || !response.ok()) throw new Error('Fetch failed');
  if (!/Hostname:/.test(await page.content()))
    throw new Error('Expected fixture content is missing');
} finally {
  await browser.close();
}
```

Simplified client running in the labelled pod. The full harness adds bounded retries and timing. Match Playwright client/server versions.

Inside the sandbox

```
import { chromium } from 'playwright';

const server = await chromium.launchServer({
  port: 9222,
  host: '0.0.0.0',
  wsPath: '/',
  args: process.env.CHROMIUM_ARGS.split(','),
});
console.log(server.wsEndpoint());
```

The image contains Node, Playwright, and Chromium.

Excerpt of the env-driven server. MCP is a separate process. Full server

Know the demo's security posture

- Agent-sandbox template uses the **cluster-default runtime**
- Chromium starts with `--no-sandbox`
- Template network-policy management is **Unmanaged**
- Browser pods do **not** mount service-account tokens

> This demonstrates the integration, not hardened hostile-code isolation.

At scale, warm is not free

Capacity

3 warm pods request 750m CPU and 1.5 GiB memory before claims.

Refill

Bursts consume ready capacity faster than new browsers may start.

Lifetime

A live WebSocket prevents idle reap, even when it is silent.

Bound sandboxes and client pods need capacity too.

03 / DEMO

Playwright inside agent-sandbox

Two WS clients. One MCP client. Three sandboxes.

Before going on stage

```
git clone https://github.com/carlossg/playwright-k8s-sandbox
cd playwright-k8s-sandbox
git checkout 5933817

bash test/harness.sh up
bash test/harness.sh e2e
```

- Requires Docker, kind, kubectl, Helm, and Go
- Match kubectl to the cluster; allow VM headroom
- Prebuild and preload the browser image

The harness creates local infrastructure and resets proxy-managed claims. Use a dedicated demo environment; do not run it on production.

Watch the claim, not just the pod

```
# Terminal 1: keep this watch running
kubectl --context kind-playwright-proxy -n pw-test \
  get sandboxclaims.extensions.agents.x-k8s.io -w
```

```
# Terminal 2: use the cached image and repeat the e2e
bash test/harness.sh e2e
```

Alpha and beta drive WS. Gamma drives MCP.

E2e intentionally clears old managed claims first; this is a fresh-assignment demo, not a warm-session timing measurement.

Prove separate sandboxes

```
kubectl --context kind-playwright-proxy -n pw-test \
  get sandboxclaims.extensions.agents.x-k8s.io \
  -o custom-columns=CLAIM:.metadata.name,SANDBOX:.status.sandbox.name

kubectl --context kind-playwright-proxy -n pw-test \
  logs job/playwright-e2e-alpha

kubectl --context kind-playwright-proxy -n pw-test \
  logs job/playwright-e2e-beta
```

Expect **different sandbox names** and successful fixture fetches.

> The target hostname can be the same: both browsers fetch the same fixture.

Show the MCP path too

```
kubectl --context kind-playwright-proxy -n pw-test \
  logs job/playwright-mcp-e2e-gamma
```

The client initializes MCP, lists tools, and calls **`browser_navigate`** against the fixture.

Gamma must have a third, distinct sandbox.

Recorded October run: 25 tools advertised; navigation returned expected fixture content. This does not prove every advertised tool was exercised.

Reuse is not restore

```
bash test/bench.sh agent-sandbox
```

Scenario	What happens
Cold	Create a new claim; use prepared capacity
Warm	Reuse the live sandbox
Script's `restore`	Delete and recreate; fresh browser

> The same ID after deletion does not recover the old browser session.

04 / EVIDENCE AND LIMITS

What actually worked?

Test the browser, not the install

Backend / path	October 16 recorded result
agent-sandbox: WS + MCP	**PASS**: separate assignments + real fixture fetch
isola: WS	**PASS**: gVisor pods + permitted cross-namespace fetch
substrate: WS	**PASS**: fetch + browser navigation after memory resume
KARS: loopback	**PASS**: WS-driven browser rendering
KARS: private fixture	**FAIL**: enforced egress boundary

Local evidence record / Source record

Local lifecycle observations

Client-observed total time, milliseconds

Backend	Cold	Warm	Final scenario
agent-sandbox	633	465	579: recreate
isola	3003	168	3455: recreate
substrate	2448	281	930: memory resume

> One observation per scenario. Not a performance ranking.

Images preloaded; shared ARM64 VM; some runs overlapped; different cluster versions. KARS has no comparable successful fixture row. Full record

Be precise about the missing proof

- Memory-fidelity resume is not an application-state oracle
- A loopback fetch is not private-network egress
- A warm pool is not an empty-node cold start
- One observation is not a latency distribution

> Report what the test establishes, not what the product name suggests.

Before calling it production-ready

- Bind tenant identity to authenticated ownership
- Validate runtime and browser sandbox settings
- Enforce and test required ingress, DNS, and egress
- Bound concurrency, lifetime, CPU, memory, and storage
- Observe assignment, pool refill, errors, and cleanup

> Scale the policy and lifecycle model, not just the replica count.

Three things to take away

> Compare lifecycle, runtime, and policy separately

> agent-sandbox makes prepared browsers easy to claim

> Prove the real browser path with enforcement enabled

Try the workload. Read the evidence.

github.com/carlossg/playwright-k8s-sandbox

Original comparison

October update and caveats

Carlos Sanchez / csanchez.org

Questions?

APPENDIX / VERSION SCOPE

What was actually pinned?

Component	Tested version
agent-sandbox	v1.0.7
isola / chart	v0.7.0 / 0.7.0
Azure KARS	v0.1.26
substrate	v0.5.0
Playwright / MCP / Node	1.53.0 / 0.0.29 / 22

Checked October 16, 2026; not a claim about versions available on the conference date. Version pins

APPENDIX / RUNTIME CHOICE

Same lifecycle, different boundary

```
# SandboxTemplate excerpt; requires configured nodes:
spec:
  podTemplate:
    spec:
      runtimeClassName: gvisor
```

Standard containers share the host kernel.

gVisor interposes a userspace kernel.

Kata uses a VM-based boundary.

> Selecting a RuntimeClass is not installing or validating its runtime.

Follow the failed boundary

Symptom	Inspect
Caller rejected	Pod label ownership, preserved source IP, informer readiness
Claim never ready	Controller conditions, template, scheduling, pool capacity
WS connects; page fails	Browser logs, DNS, target readiness, enforced egress
MCP fails; WS works	MCP process, 9223 routing, protocol/version compatibility
Idle resource never reaped	Active WebSocket count, TTL, client cleanup

> Fix the narrow integration contract; do not disable the boundary to hide the failure.

APPENDIX / REPRODUCE OTHER BACKENDS

Same browser, different backend

```
bash test/harness.sh up-isola
bash test/harness.sh test isola

bash test/harness.sh up-substrate
SUBSTRATE_FLAVOUR=playwright bash test/harness.sh test substrate

bash test/harness.sh up-kars
bash test/harness.sh test kars
# The original private fixture currently fails under enforced egress.
```

Run from the pinned source checkout. Substrate has additional prerequisites and its own cluster; follow the deployment guide.